

A decorative graphic consisting of blue lines and small circles, resembling a circuit board or a network diagram, extending horizontally across the middle of the slide. The lines are composed of straight segments and right-angle turns, with small circles at the endpoints and intersections.

SISTEMAS OPERACIONAIS

PROFESSOR MARCOS MENDES



INTRODUÇÃO AOS SISTEMAS OPERACIONAIS

Um **Sistema Operacional (SO)** é um software responsável por gerenciar os recursos do hardware e fornecer serviços para os programas.



FUNÇÕES PRINCIPAIS DO S.O

1. Gerenciamento de processos

- Controla quais programas estão executando

2. Gerenciamento de memória

- Controla uso da RAM

3. Gerenciamento de arquivos

- Organiza dados no disco

4. Gerenciamento de dispositivos

- Controla teclado, mouse, disco, rede



EVOLUÇÃO HISTÓRICA DOS S.O

1ª Geração (1940–1956) — Sem sistema operacional

- Computadores gigantes
- Programação manual
- Sem interface
- Um programa por vez



EVOLUÇÃO HISTÓRICA DOS S.O

Os computadores de primeira geração conseguiam realizar milhares de cálculos por segundo, mas só podiam realizar uma operação por vez e consumiam muita energia elétrica.

Eles foram programados em linguagem de máquina, uma linguagem de programação de baixo nível, e a entrada e saída dos dados era feita a partir de cartões perfurados.



HARVARD MARK I, 1943 (SEM S.O)



EVOLUÇÃO HISTÓRICA DOS S.O

2ª Geração (1950–1960) — Sistemas em lote (Batch)

- Uso de cartões perfurados
- Processamento em lote
- Sem interação com usuário



EVOLUÇÃO HISTÓRICA DOS S.O

Os computadores de segunda geração não se diferenciaram apenas pela tecnologia e pelo menor tamanho, mas pela mudança na linguagem de programação, que passou para a linguagem *assembly*.

PDP-1





EVOLUÇÃO HISTÓRICA DOS S.O

3ª Geração (1960–1980) — Multiprogramação

- Vários programas na memória
- Surgimento de **time-sharing**
- Melhor uso da CPU



EVOLUÇÃO HISTÓRICA DOS S.O

Caracterizada pela incorporação de circuitos integrados que substituíram os transistores.

Nesse tipo de computador, os dados de entrada e saída eram gerenciados por dispositivos periféricos como monitor, teclado ou impressora.



IBM 360, utilização de circuitos integrados - terceira geração de computadores



EVOLUÇÃO HISTÓRICA DOS S.O

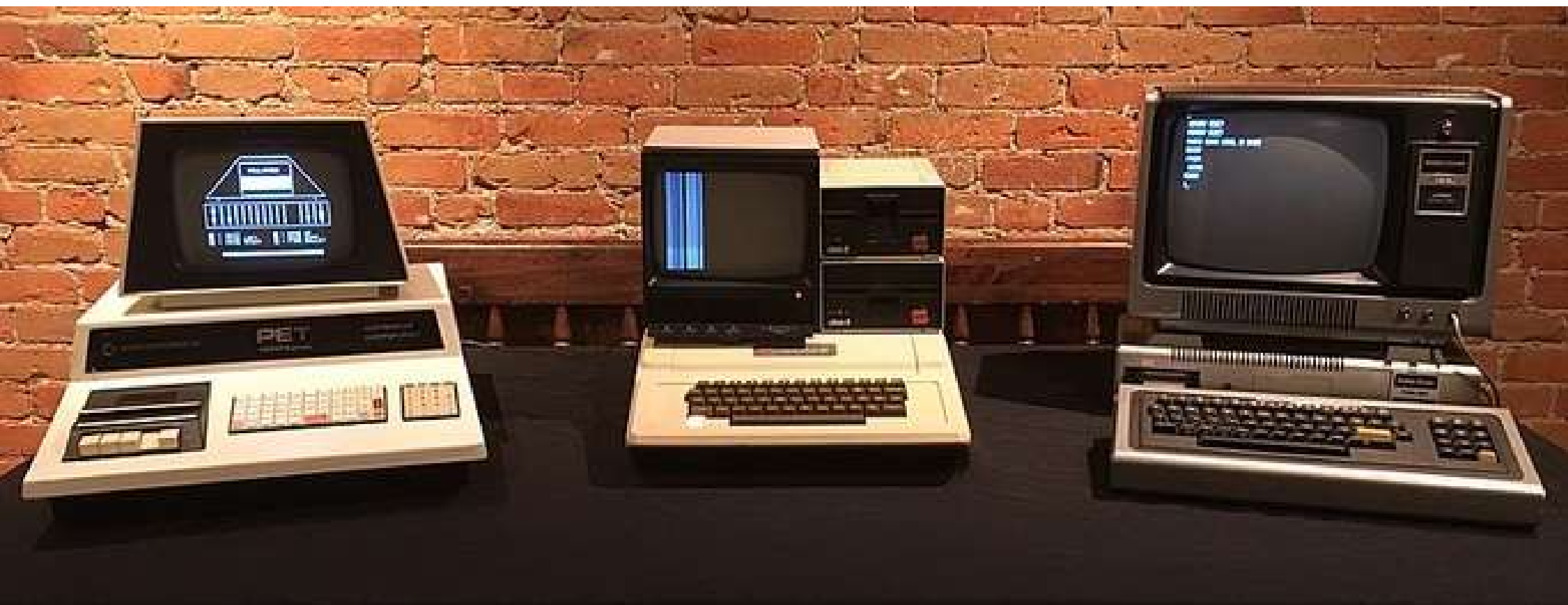
4ª Geração (1980—hoje) — Sistemas modernos

- Computadores pessoais
- Interfaces gráficas
- Sistemas distribuídos e cloud
- Linux, Windows, macOS



EVOLUÇÃO HISTÓRICA DOS S.O

A partir de 1971, os computadores deixaram de funcionar com circuitos integrados e incorporaram os microprocessadores.



Da esquerda para a direita: Commodore PET 2001, Apple II e TRS-80 Model I - computadores de quarta geração (1977)



EVOLUÇÃO HISTÓRICA DOS S.O

5ª Geração (Presente-Futuro) - Inteligência Artificial:

Focada em sistemas operacionais móveis, distribuídos, nuvem e com recursos de inteligência artificial, reconhecimento de voz/facial e computação quântica.

The image shows the interior of a quantum computer, featuring a complex arrangement of gold-colored metal components, including vertical rods and horizontal rings, all housed within a dark, reflective enclosure. A dense network of white cables is visible at the bottom, connected to a series of gold-colored connectors. A black rectangular box with a thin green border is centered over the image, containing white text. On the left and right sides of this box, there are stylized green circuit traces with small circles at their ends, resembling electronic components or data paths.

INTERIOR DE UM
COMPUTADOR QUÂNTICO.



EVOLUÇÃO HISTÓRICA DOS S.O

- Computadores quânticos utilizam princípios da mecânica quântica, como superposição e emaranhamento, para processar informações de forma exponencialmente mais rápida que computadores clássicos.

“O chip Willow do Google, por exemplo, realizou em 5 minutos um cálculo que supercomputadores tradicionais levariam anos (10 septilhões) para finalizar.”



OBJETIVOS DO SISTEMA OPERACIONAL



OBJETIVOS DO SISTEMA OPERACIONAL

Eficiência

- Usar bem os recursos do sistema
- Evitar CPU ociosa
- Gerenciar memória corretamente



OBJETIVOS DO SISTEMA OPERACIONAL

Capacidade de evolução

- Permitir melhorias e atualizações
- Atualizações do sistema
- Novos drivers



OBJETIVOS DO SISTEMA OPERACIONAL

4. Segurança

- Proteger dados e usuários
- Permissões
- Autenticação



TESTE CONCEITUAL

1. O que é um sistema operacional?

- a) Um hardware
- b) Um programa que gerencia recursos
- c) Um dispositivo de entrada
- d) Um compilador



TESTE CONCEITUAL

2. Qual das alternativas NÃO é função de um sistema operacional?

- a) Gerenciar memória
- b) Controlar dispositivos
- c) Executar diretamente código binário sem controle
- d) Gerenciar processos



TESTE CONCEITUAL

3. O sistema operacional atua como:

- a) Um aplicativo comum
- b) Intermediário entre usuário e hardware
- c) Apenas interface gráfica
- d) Apenas sistema de arquivos



ESTRUTURA DOS SISTEMAS OPERACIONAIS

Camadas:

- Hardware
- Kernel
- Serviços do sistema
- Aplicações
- Usuário



KERNEL — O CORAÇÃO DO SISTEMA

O **Kernel** é a parte central do sistema operacional responsável por controlar o hardware.

Funções do Kernel

- Gerenciar processos
- Gerenciar memória
- Controlar dispositivos
- Gerenciar sistema de arquivos
- Controlar acesso ao hardware



TIPOS DE KERNEL

Monolítico

- Tudo dentro do kernel
- Ex: Linux

Microkernel

- Kernel mínimo
- Serviços no espaço do usuário

Híbrido

- Mistura dos dois
- Ex: Windows



SHELL — INTERFACE COM O USUÁRIO

O **Shell** é a interface entre o usuário e o sistema operacional.

Tipos

- CLI (Terminal)
- GUI (Interface gráfica)



MODOS DE OPERAÇÃO

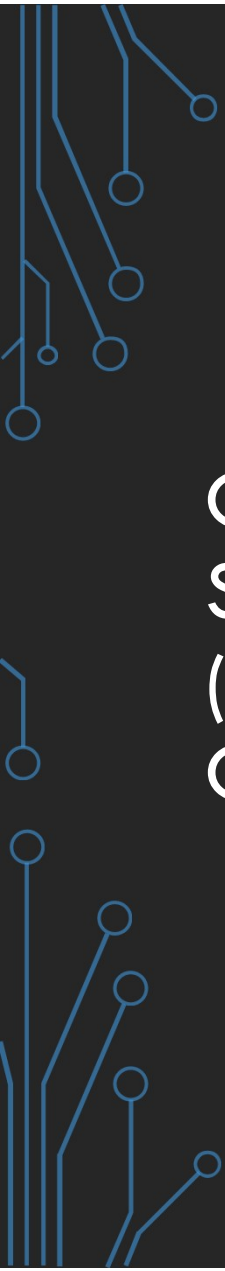
Dois modos principais

Modo Kernel (privilegiado)

- Acesso total ao hardware
- Executa código crítico

Modo Usuário

- Acesso limitado
- Aplicações comuns



CHAMADAS DE SISTEMA (SYSTEM CALLS)

System calls são a interface entre programas e o kernel.

Exemplos

- abrir arquivo
- criar processo
- ler dados
- escrever dados



CONCEITO, ESTADOS, PCB E TROCA DE CONTEXTO

Conceito de Processo

Um **processo** é um programa em execução.

- notepad.exe → programa
- Notepad aberto → processo

Um mesmo programa pode ter vários processos!

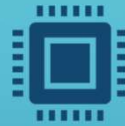
ESTADOS DE UM PROCESSO



NEW - Processo está sendo criado.



Ready - Pronto para executar, aguardando CPU.



Running - Usando a CPU.



Waiting - Esperando evento (I/O, por exemplo).



Terminated - Execução terminou



PCB — PROCESS CONTROL BLOCK

O **PCB** é a estrutura onde o sistema operacional guarda informações do processo. (ficha do processo no sistema).

O que o PCB contém?

- PID (ID do processo)
- Estado do processo
- Registradores da CPU
- Contador de programa
- Informações de memória
- Informações de arquivos abertos



TROCA DE CONTEXTO (CONTEXT SWITCH)

Troca de contexto é quando o sistema operacional pausa um processo e executa outro.

TROCA DE CONTEXTO (CONTEXT SWITCH)

Como funciona?

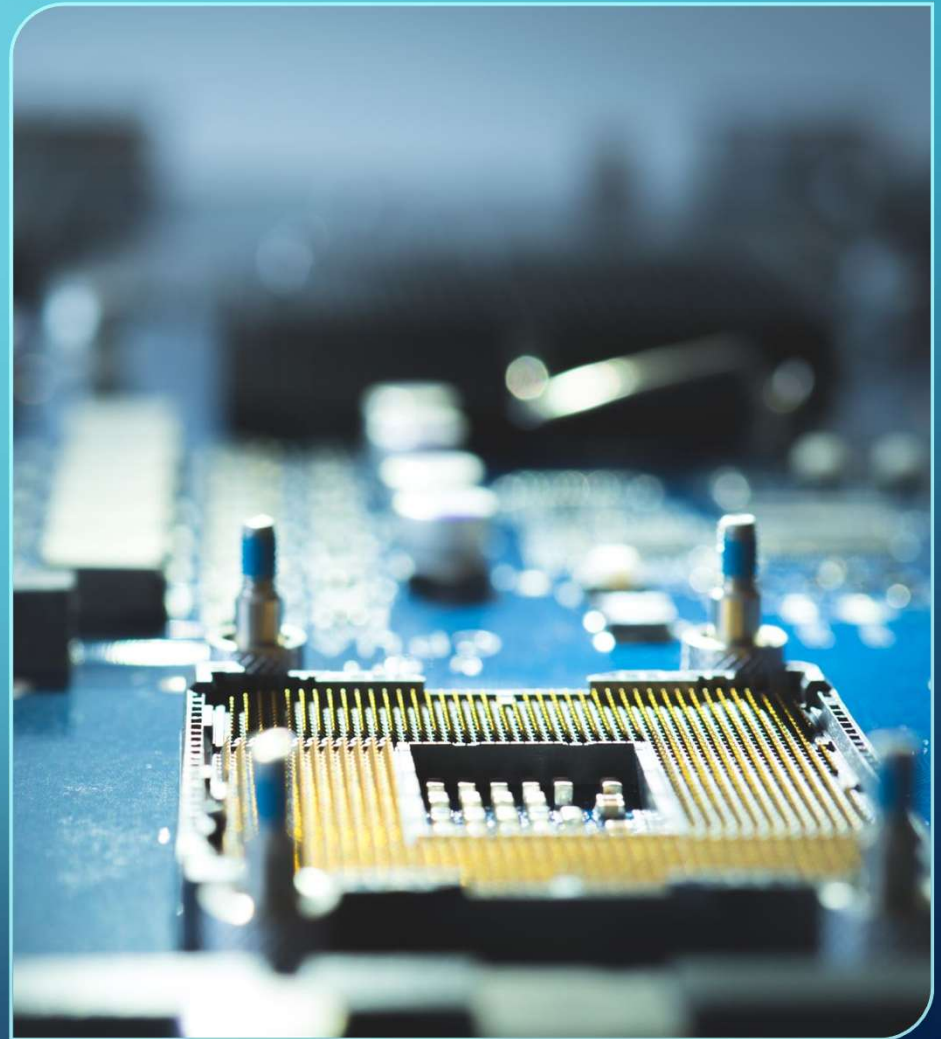
- Salva estado do processo atual (PCB)
- Carrega estado do próximo processo
- CPU continua execução

Por que isso é necessário?

- Multiprogramação
- Vários processos compartilhando CPU

Ponto importante

- Troca de contexto tem custo!
- não é “de graça”
- impacta performance





ESCALONAMENTO DE PROCESSOS

Escalonamento é o processo de decidir qual processo utilizará a CPU em determinado momento.

Por que isso é necessário?

- CPU é recurso limitado
- Muitos processos competindo
- Precisamos organizar



CONCEITO DE ESCALONAMENTO

Objetivos do escalonamento

- Maximizar uso da CPU
- Minimizar tempo de espera
- Garantir justiça
- Evitar starvation



MÉTRICAS IMPORTANTES

Tempo de espera

- Tempo que o processo fica aguardando na fila

Tempo de retorno (Turnaround)

- Tempo total do processo:
- chegada → finalização

Tempo de resposta

- Tempo até o processo começar a executar

ALGORITMO FCFS (FIRST COME FIRST SERVED)

Processo	Burst
P1	5
P2	3
P3	2

Execução

P1 → P2 → P3

Vantagens

- Simples
- Fácil de implementar

Problemas

- Efeito comboio (processos pequenos esperam muito)

ALGORITMO SJF (SHORTEST JOB FIRST)

Processo	Burst
P1	5
P2	3
P3	2

Execução

P3 → P2 → P1

Vantagens

- Menor tempo médio de espera

Problemas

- Pode causar starvation
- Difícil prever tempo de execução

ROUND ROBIN (RR)

Cada processo recebe um tempo fixo (quantum)

Processo	Burst
P1	5
P2	3
P3	2

Execução (cíclica)

P1 → P2 → P3 → P1 → P2 →
P1

Vantagens

- Justo
- Ideal para sistemas interativos

Problemas

- Muitas trocas de contexto

ESCALONAMENTO POR PRIORIDADE

Cada processo tem uma prioridade!

Execução

- Maior prioridade → executa primeiro

Problema

- Starvation (processos de baixa prioridade nunca executam)

Solução

- Aging (aumenta prioridade com o tempo)



SINCRONIZAÇÃO DE PROCESSOS

“Se dois processos acessarem o mesmo dado
ao mesmo tempo... o que pode acontecer?”



CONCORRÊNCIA E CONDIÇÃO DE CORRIDA

Condição de corrida ocorre quando dois ou mais processos acessam dados compartilhados simultaneamente, causando resultados imprevisíveis.



REGIÃO CRÍTICA

- Região crítica é a parte do código onde ocorre acesso a recursos compartilhados.



REGIÃO CRÍTICA

Regra fundamental

- Apenas **um processo por vez**



REGIÃO CRÍTICA

Objetivos

- Exclusão mútua
- Evitar inconsistência
- Garantir integridade

MUTEX (MUTUAL EXCLUSION)

Mutex é um mecanismo que permite apenas um processo acessar a região crítica.

Banheiro com chave  “Só uma pessoa entra por vez”

SEMÁFOROS

Semáforo controla o acesso a recursos com múltiplas permissões.

<> Python



▶ Executar

```
semaforo = Semaphore(2)
```

👉 até 2 processos simultâneos

Banheiro com algumas cabines 🚻

“Uma pessoa entra por vez”

DEADLOCK (TRAVAMENTO)

Deadlock ocorre quando processos ficam esperando uns pelos outros indefinidamente.

Exemplo clássico

- Processo A segura recurso 1 e quer recurso 2
- Processo B segura recurso 2 e quer recurso 1

👉 Ninguém continua 🤯

Dois carros numa rua estreita 🚗 🚗

Nenhum consegue passar → deadlock

condições do deadlock

1. Exclusão mútua
2. Hold and wait
3. Não preempção
4. Espera circular

Soluções

- Evitar
- Prevenir
- Detectar
- Recuperar



GERÊNCIA DE MEMÓRIA

“POR QUE UM SISTEMA TRAVA QUANDO A
MEMÓRIA ACABA?”

O QUE É GERÊNCIA DE MEMÓRIA?

É a função do sistema operacional responsável por gerenciar o uso da memória principal (RAM).

Responsabilidades do SO

- Controlar uso da memória
- Alocar memória para processos
- Liberar memória
- Evitar conflitos
- Otimizar uso

Cada processo acha que tem memória “só pra ele”



MEMORIA RAM

Características

- Volátil
- Rápida
- Limitada

Importante

Todos os processos competem por RAM

ALOCAÇÃO DE MEMÓRIA

Ação de reservar espaço na memória para execução de processos.

Alocação Contígua

- Processo ocupa bloco contínuo
- Simples, mas ineficiente

Partições Fixas

- Memória dividida previamente
- Pode desperdiçar espaço

Partições Variáveis

- Aloca conforme necessidade
- Mais eficiente

ESTRATÉGIAS DE ALOCAÇÃO

First Fit

👉 Primeiro espaço disponível

Best Fit

👉 Melhor encaixe

Worst Fit

👉 Maior espaço disponível



Como guardar caixas em um armário

FRAGMENTAÇÃO

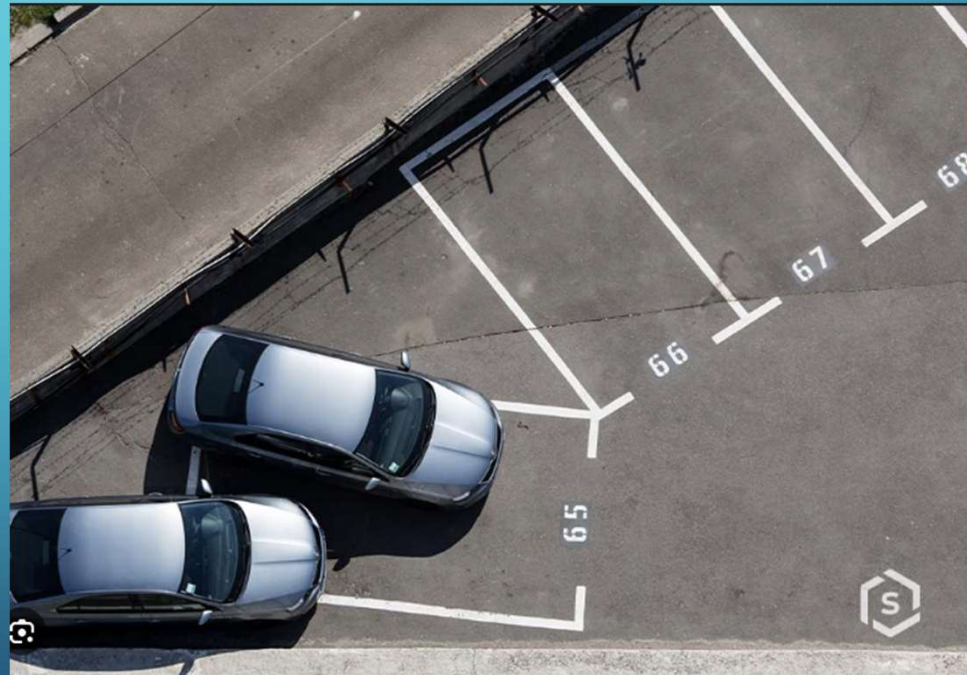
Fragmentação ocorre quando a memória fica mal aproveitada.

Fragmentação Interna

- Espaço sobrando dentro do bloco

Fragmentação Externa

- Espaços livres espalhados



SWAPPING

Técnica de mover processos da memória RAM para o disco (swap)

Por que usar?

- RAM limitada
- Muitos processos

Problema

Disco é MUITO mais lento

Consequência

- Lentidão
- Sistema “travando”

Quando o sistema começa a usar swap intensivamente...

👉 é sinal de problema de memória

👉 muito comum em:

- servidores mal dimensionados
- containers sem limite
- aplicações pesadas

MEMÓRIA VIRTUAL

Memória virtual é uma técnica que permite ao sistema operacional usar mais memória do que a RAM física disponível

Ideia principal

O processo acha que tem memória “infinita” Mas o SO está “enganando” ele...

Como funciona

- Parte na RAM
- Parte no disco (swap)
- SO gerencia tudo

Benefícios

- Executar programas grandes
- Isolamento entre processos
- Melhor uso da memória



PAGINAÇÃO

A paginação é uma técnica que divide a RAM e os processos em blocos de tamanho fixo (páginas/frames), eliminando a fragmentação externa.

O SO usa uma "page table" para mapear endereços virtuais na RAM física, transferindo dados inativos para um "arquivo de paginação" (swap) no disco.

PAGINAÇÃO

Divide a memória em blocos fixos chamados páginas.

Conceitos

- Página → memória virtual
- Frame → memória física

Funcionamento

- Processo dividido em páginas
- RAM dividida em frames
- Mapeamento feito por tabela de páginas

Vantagens

- Elimina fragmentação externa
- Flexível

Desvantagens

- Overhead (tabela de páginas)



SEGMENTAÇÃO

A segmentação de memória RAM é uma técnica que divide o espaço de endereçamento em segmentos lógicos de tamanhos variáveis, como código, dados e pilha.

SEGMENTAÇÃO

Divide memória em segmentos lógicos (código, dados, pilha).

Características

- Tamanhos variáveis
- Baseado em lógica do programa

Problema

Fragmentação externa

Exemplo

- Segmento código
- Segmento dados
- Segmento stack

PAGE FAULT

Ocorre quando um processo tenta acessar uma página que não está na RAM.

Fluxo

1. Processo solicita página
2. Página não está na RAM
3. SO busca no disco
4. Carrega para memória

Consequência

- 👉 Lento (acesso a disco)

Importante

- 👉 Page fault é normal
- 👉 Problema é excesso

THRASHING

Thrashing ocorre quando o sistema passa mais tempo trocando páginas do que executando processos.

Sintomas

- CPU baixa
- Disco alto
- Sistema lento

Causa

👉 Falta de memória

Muito comum em:

- Servidores mal dimensionados
- Containers sem limite de memória
- Aplicações mal otimizadas

ORGANIZAÇÃO EM DISCO

Conceito

O disco é dividido em:

- blocos
- setores
- regiões

O SO gerencia:

- onde armazenar dados
- como recuperar
- como evitar desperdício

SISTEMAS DE ARQUIVOS

Sistema de arquivos é a forma como o sistema operacional organiza, armazena e recupera dados no disco.

Funções

- Armazenar dados
- Organizar arquivos
- Controlar acesso
- Gerenciar espaço em disco

Exemplos

- ext4 (Linux)
- NTFS (Windows)
- FAT32

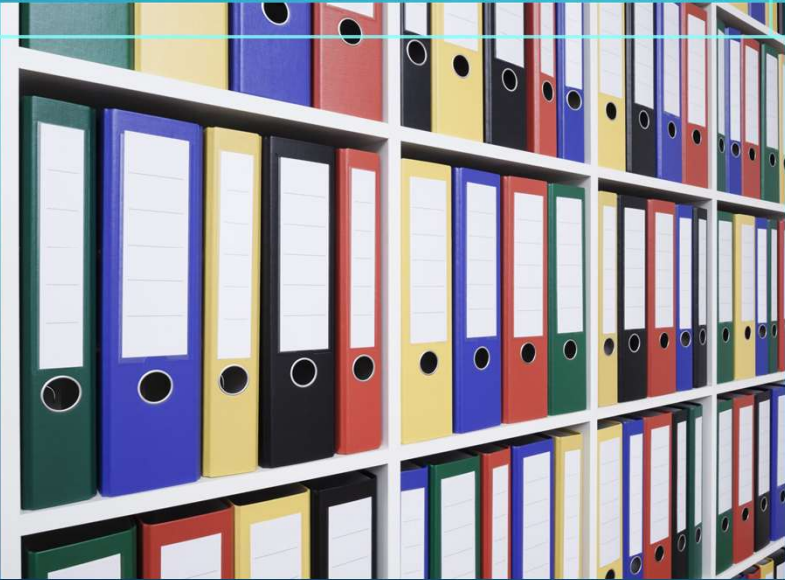
O sistema de arquivos é como um “mapa do disco”

Sem ele:

- dados se perdem
- não há organização

Arquivo

Unidade básica de armazenamento



SISTEMAS DE ARQUIVOS

- **Diretório**
- Estrutura que organiza arquivos



WINDOWS VS LINUX

A principal diferença entre os sistemas de arquivos Linux e Windows está na estrutura e organização.



LINUX

Linux usa uma estrutura de árvore única (raiz /) onde tudo é um arquivo.

Linux suporta nativamente formatos como ext4, XFS e Btrfs, focados em performance.

Começa no diretório raiz (/). Todos os discos, partições e dispositivos são "montados" como subpastas (ex: /mnt/dados).

Diferencia maiúsculas de minúsculas
(ex: Arquivo.txt e arquivo.txt são arquivos diferentes).

Usa barra para a frente (/), ex: /home/usuario/documentos.



WINDOWS

Windows organiza dados por unidades distintas (C:, D:).

Windows usa NTFS, otimizado para compatibilidade.

Usa letras de unidade para separar discos físicos ou partições (ex: C:\, D:\).

Geralmente não diferencia maiúsculas de minúsculas (ex: Arquivo.txt e arquivo.txt são considerados o mesmo arquivo).

Usa barra invertida (\), ex: C:\Users\Usuario\Documentos.



SISTEMAS DE ENTRADA E SAÍDA (I/O)

“Quando você digita no teclado... como esse dado chega até o sistema?”



SISTEMAS DE ENTRADA E SAÍDA (I/O)

I/O (Entrada/Saída ou *Input/Output*) em sistemas operacionais refere-se à comunicação entre o processador/memória e dispositivos externos (periféricos).

O QUE É I/O (ENTRADA E SAÍDA)

I/O é a comunicação entre o sistema computacional e o mundo externo.

Exemplos

- Teclado → entrada
- Mouse → entrada
- Disco → leitura/escrita
- Tela → saída
- Rede → entrada/saída

O sistema operacional gerencia essas operações, utilizando controladores de dispositivos e drivers para traduzir comandos de software em ações físicas, lidando com interrupções e gerenciando o buffer de dados.

DISPOSITIVOS E DRIVERS

Driver

Driver é o software que permite ao sistema operacional controlar o dispositivo.

Exemplo

- driver de rede
- driver de vídeo
- driver de disco

SO não fala diretamente com hardware, fala através do driver

Dispositivos

- Entrada: teclado, mouse
- Saída: monitor
- Entrada/Saída: disco, rede





SEGURANÇA EM SISTEMAS OPERACIONAIS

“Se qualquer usuário pudesse acessar qualquer arquivo do sistema... o que aconteceria?”



CONCEITOS DE SEGURANÇA

Objetivos principais

- Confidencialidade
- Integridade
- Disponibilidade



CONCEITOS DE SEGURANÇA

Confidencialidade → quem pode ver

Integridade → dados corretos

Disponibilidade → sistema funcionando



AUTENTICAÇÃO VS AUTORIZAÇÃO

Autenticação

- Verificar quem você é, ou melhor, quem é você?

(login + senha)

Autorização

- O que você pode fazer
- (permissões)



PERMISSÕES NO SISTEMA

Tipos

- leitura (r)
- escrita (w)
- execução (x)

LOGS DO SISTEMA

Logs são registros de eventos do sistema.

Para que servem?

- auditoria
- debug
- segurança

HARDENING

Hardening é o processo de fortalecer a segurança do sistema.

Exemplos

- remover serviços desnecessários
- atualizar sistema
- usar firewall
- restringir permissões
- usar autenticação forte

Princípio importante

Menor privilégio



REFERÊNCIAS

- SILBERSCHATZ, Abraham; GALVIN, Peter; GAGNE, Greg.
Fundamentos de Sistemas Operacionais. LTC.
- TANENBAUM, Andrew; BOS, Herbert.
Sistemas Operacionais Modernos. Pearson.
- MACHADO, Francis; MAIA, Luiz.
Arquitetura de Sistemas Operacionais. LTC.



REFERÊNCIAS

- STALLINGS, William.
Sistemas Operacionais: Internos e Projeto. Pearson.
- NEMETH, Evi et al.
Unix and Linux System Administration Handbook.

Referências online

- Documentação oficial Linux
- man pages (man ps, man top, etc.)
- <https://kernel.org>